



NajVR: low-latency plug-and-play remote rendering for interactive virtual reality applications

José A. Santos-Paz¹ · José J. García Aranda² · Manuel Alarcón Granero³ · Francisco J. J. Quintanilla⁴ · Pablo Perez-Tirador¹ · Abraham Otero¹ · Rodrigo García-Carmona⁵

Received: 6 August 2025 / Accepted: 10 November 2025 / Published online: 20 November 2025
© The Author(s) 2025

Abstract

Virtual Reality (VR) setups connected to a computer offer a more realistic experience compared to standalone devices. However, these setups require high-performance hardware, making them expensive and less portable. A more cost-effective and portable alternative involves leveraging shared cloud or edge resources to render VR experiences. This work introduces NajVR, an architecture for remote rendering of VR applications that requires only commodity hardware on the client and includes LHE, a codec designed to minimize latency and resistant to packet loss that uses the user's gaze data to improve the encoding process. NajVR does not require any modifications to the VR applications or their runtime environment. In NajVR an application on the client captures pose data and events generated by the VR hardware and transmits them to the server. On the server, a driver processes these data, simulating the VR hardware as if it were connected directly to it. This driver also captures the images to be displayed on the Head Mounted Display (HMD), encodes them, and transmits them to the client. NajVR supports an arbitrary number of trackers and takes advantage of the user's gaze data to improve the perceived quality. The architecture round-trip latency was measured, and the LHE codec was compared with four different configurations of two commercially available codecs. The tests were carried out on both high-bandwidth and resource-constrained networks and in WAN setups. In the best-case scenario, latencies of just over 20 ms were achieved.

Keywords Virtual reality · Remote rendering · Cloud gaming · Video encoding · Low-latency · Eye tracking

1 Introduction

Virtual reality (VR) experiences are delivered through two main types of setups: standalone all-in-one headsets and PC-based systems. While standalone devices offer portability and affordability, they lack the computational power of PC VR, which can generate more complex and realistic virtual environments. One way to strike a balance between the two approaches is to connect a standalone device to a computer and split the computing tasks between them. This also enables the use of cloud- or edge-based resources. Several implementations of this core idea have been explored in the literature.

One strategy is to make the standalone device render each frame with reduced detail while a remote server sends a stream of differences per frame to transform each frame into a high-detail version [1]. The problem with this approach is that the differences per frame make reference to a previous frame and might not map properly to the one that needs to be rendered.

Other techniques include motion prediction combined with image deflection [2] or warping [3–5]. While successful for head position and rotation, these are generally limited to static environments. Although prediction can still track controllers, their 3D representation makes warping challenging due to potential occlusion issues. Crucially, none of these methods effectively handle the display of the user’s virtual body.

A different method is split rendering [6–8], where the background and non-interactive objects are rendered remotely and the foreground objects locally. Issues include the difficulty of managing object transitions between foreground/background and tends to produce environments with two very distinct levels of detail. Alternatives involve the server sending depth information for client-side processing [9] or using layer-based encoding with client-side caching [10]. Given that illumination is the most demanding part of the rendering pipeline, some researchers partition the shading load: the server computes global illumination effects (ray-tracing), and the client renders simpler local illumination [11, 12] effects. A common problem with split rendering is the fidelity mismatch between local and remote objects, and “pop-in” effects when new server data arrives.

A pervasive issue in all split rendering is the misalignment between server-rendered data and local data due to orientation and position differences. Some solutions [7, 8] address this by having the server render and stream the entire 360° area around the player. This technique, also used without split rendering [13, 14] with a slightly expanded field of view (FoV), lets local hardware select the best subset of the received image based on the user’s latest orientation. However, this only compensates for rotation changes, not position, and is computationally more expensive for the server due to the larger required FoV.

A promising approach to bridge this gap is remote rendering [15, 16] where a powerful computer (either a local PC or a cloud-based server) handles the demanding rendering tasks and streams the resulting video to a lightweight standalone headset. This strategy eliminates most problems related to the synchronization and harmonization of remote and local contents. Several authors have chosen different ways to reduce latency: by pipelining the rendering process [15], by reducing the amount of memory copies that are needed to render, encode, and stream content [16], or by

dynamically adjusting how the stream is encoded based on its characteristics [17]. A problem common to the solutions presented in the literature is that they require extensive modifications in the game engine, the operating system, the network stack, and/or the VR devices' drivers.

However, there are several commercial solutions that can stream unmodified VR applications. Systems like Virtual Desktop and Shadow PC are capable of streaming PC VR games to standalone headsets. Virtual Desktop is a mature solution that supports wireless streaming and can forward advanced sensor data, such as eye and facial tracking if the device supports it. NVIDIA's CloudXR is another established technology in this space, designed for streaming from remote servers over both LAN and WAN connections. On the open-source front, ALVR provides another popular solution for streaming VR content. While initially presented as a LAN-only system, ALVR also supports streaming over the internet via VPN or NAT traversal.

However, many of these existing solutions, while effective, still present opportunities for improvement—particularly in latency reduction, codec efficiency, and the seamless integration of emerging hardware capabilities such as eye tracking. Among the commercial systems mentioned above, only Virtual Desktop and ALVR forward eye-tracking data, and they do not leverage this information to enhance perceived quality or the encoding process.

We introduce NajVR, an architecture designed to address these challenges. NajVR supports low-latency, cloud-based rendering of unmodified VR applications and can leverage gaze data to prioritize perceptual quality. It integrates LHE, a latency-oriented video codec that uses eye-tracking information to guide encoding decisions. In our tests, LHE has shown good tolerance to packet loss, which helps maintain stability under uneven network conditions.

The NajVR framework consists of a lightweight Dummy VR Application running on the local device (see Fig. 1), which captures and transmits user input (head/hand poses, controller inputs, and gaze data) to a cloud server. On the server, a Virtual Driver emulates the VR hardware, feeding this input to an unmodified VR application. The application's rendered frames for both eyes are then captured by the Virtual Driver, encoded using the LHE codec, and streamed back to the HMD (Head Mounted Display) for decoding and display. This architecture allows NajVR to function seamlessly with existing VR software while introducing significant performance enhancements through its gaze-aware streaming pipeline.

This approach allows for off-the-shelf graphically intensive VR experiences to be rendered remotely without requiring high-performance local hardware and with low latencies and improved perceived quality. The main contributions of this article are:

- An architecture, NajVR, that enables the rendering of off-the-shelf interactive VR applications in the cloud and leverages eye-tracking data to improve streaming.
- The LHE codec, which has been designed to reduce latency and improve robustness. This codec utilizes eye-tracking data to enhance its encoding performance.
- An implementation of the NajVR architecture and performance metrics for each of its components, including the LHE codec, which is compared to other commercial codecs in terms of bandwidth, latency, and tolerance to packet loss.

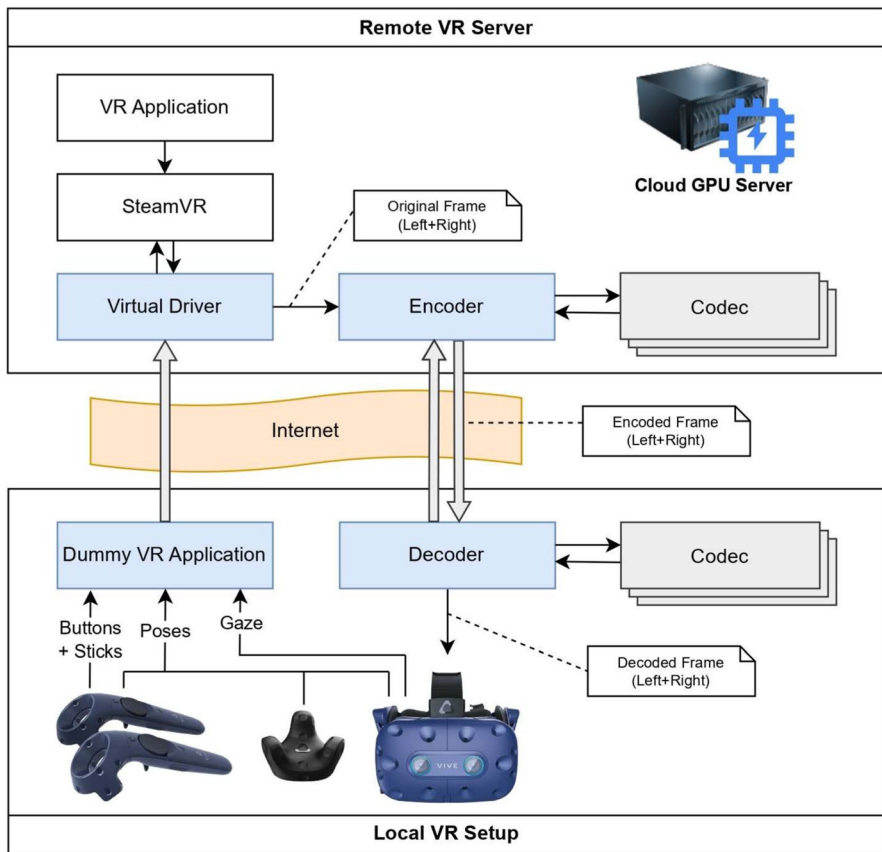


Fig. 1 System architecture. Squares with a blue background are the NajVR components that allow the remote rendering of unmodified VR applications. Squares with a white background are the (unmodified) components used in a typical VR setup. Several GPU-run codecs (in grey) can be used

The structure of the document is as follows. Section 2 provides an overview of the NajVR architecture and explains its components. Section 3 details the setup and data exchange process between the server and the client. Section 4 describes the remote rendering workflow. Section 5 focuses on the LHE codec. Section 6 presents the benchmark results of LHE compared to other codecs, as well as the performance metrics of the implementation of the different components of the NajVR architecture. Finally, Sect. 7 discusses the results of these tests and suggests potential future improvements and limitations.

2 Architecture overview

NajVR enables the remote execution of off-the-shelf VR applications, without modifying its code or performing other adaptations. In this architecture, which can be seen in Fig. 1, the rendering is performed by a remote VR server, but the inputs to the

scene rendered are retrieved from a local HMD and the resulting image is displayed onto its screen. To achieve this, NajVR has four components:

- A Dummy VR Application that runs on the client VR setup. This component retrieves the pose data of all the connected VR devices, the state of the buttons and joysticks of the controllers and, if the HMD provides this information, the points on the screen at which the user is looking at. It sends these data to a remote Virtual Driver.
- A Virtual Driver that runs on the remote VR server. This component is a SteamVR device driver that enables the server to run without any VR device connected. Instead of capturing the information of a physical device, this driver relays to the SteamVR platform the client data that was retrieved by the Dummy VR Application. After processing these data by the VR application, SteamVR delivers to this driver a set of left and right images to display. Both images are combined into one frame that is handed, together with the user's gaze data (if available), to the Encoder component instead of being displayed in an HMD.
- An Encoder that runs on the remote VR server. This component gets the frame provided by the Virtual Driver and (if available) the user's gaze information. Then, using a GPU-run codec, encodes the frame and sends it to a Decoder running on the client. The Encoder's default codec is LHE, a newly developed low-latency and robust codec that takes advantage of the user's gaze data to improve the efficiency of the image encoding strategy.
- A Decoder that runs on the client VR setup. This component receives the encoded frame created by the Encoder in the remote server, decodes it, and draws the corresponding left and right images into the physical HMD's screen.

With this setup, the VR application that is running on the remote server does not need to be modified, since all its interactions occur with the APIs provided by a stock SteamVR installation. SteamVR was selected as the platform since it supports both OpenXR and OpenVR, the two most widespread VR frameworks, and can also run Oculus-exclusive applications through the Revive compatibility layer.

3 Setup and devices data exchange

To setup the connection between the client HMD and the remote VR server, the former must send to the latter a handshake message with the characteristics of the physical VR devices being used. This information allows the Virtual Driver to emulate the client hardware in the server. Therefore, SteamVR can operate as if the peripherals were directly connected to the server. This message is sent by the Dummy VR Application.

The handshake message distinguishes between three types of VR devices:

- The HMD, of which there must always be exactly one. The native rendering resolution, the display frequency, and whether the HMD supports the tracking of the user's gaze are all specified. The current interpupillary distance (IPD) is also

provided.

- Up to two controllers, which are in the user's hands. Each controller can be defined to have as many buttons and/or joysticks as needed. Buttons can be purely binary (pushed/not pushed), analog one-dimensional triggers, or a combination of both (some triggers have a boolean activator at the end of their range). Joysticks are two-dimensional analog devices that can sometimes also be pushed like buttons. Each button and joystick has a name.
- As many tracked devices as wanted. These devices are any physical object whose position and orientation can be tracked by the VR setup.

All devices are assigned an ID, which is a single character, and have a model name. The Virtual Driver component can recognize the model name of the devices and pass it along to SteamVR; this way, the VR application can render the most appropriate shape on top of the virtual environment.

The Virtual Driver has to simulate the presence of the appropriate VR devices on the remote server, enabling the SteamVR platform to operate as if they were directly connected to the server. SteamVR looks for a connected HMD every time it is launched, going through the installed device drivers, one of each being the Virtual Driver.

Within each HMD device driver, there is a *devicesConnected* variable that is set to "true" when a compatible HMD is detected. This happens when the Virtual Driver and the Dummy VR Application have completed the handshake process. At this point, SteamVR selects that driver and uses it to communicate with the HMD.

Once the HMD is detected, an internal check is performed to make sure that the HMD's sensors and components are working properly and provide accurate data about the user's position and movement in the virtual environment. Once configured and connected, SteamVR displays a graphical interface on the user's screen, providing information on the status of the HMD, the controllers, and any tracked device (if present) and allows the user to access the library of available VR applications from inside SteamVR itself. As part of this configuration, the user might need to define the boundaries of the area in which they will move; a process called "room setup" that is physically performed in the client's location, where the real HMD is located.

The Virtual Driver, which is accessed via a dynamic link library (DLL), provides the data of each device to SteamVR through the function *TrackedDevicePoseUpdated*, and informs about changes in the state of any control input through the *VRDriverInput* function, using *UpdateBooleanComponent* for true and false events, and *UpdateScalarComponent* for analog events.

Once the handshake process has been completed and SteamVR has finished booting up, the Dummy VR Application starts collecting the state of the devices defined in the handshake message. These data are retrieved at the display frequency of the HMD, and parsed immediately after every frame is rendered.

For every tick of the display frequency (usually between 72 and 120 Hz), the Dummy VR application prepares the collected data and sends them to the Virtual Driver using UDP (port 54000 by default). This protocol is ideal for a real-time application, since messages do not need to be acknowledged and the data sent are always absolute, with no reference to a previous frame's data.

The way in which the data are packaged is shown in Fig. 2. It always begins with the HMD data, followed by the controllers (as many as defined during the handshake), and finishes with the tracked devices (again, as many as defined).

Each device begins with the ID (*char*, 1 byte) defined during the handshake process. It is followed by the device's *pose*. The pose is defined as the combination of the position in the X, Y, and Z coordinates (three double-precision *floats*, 8 bytes each), with the Z-axis pointing front and a left-handed coordinate system; and the rotation with qX, qY, qZ, and qW defining a quaternion (four double-precision *floats*, 8 bytes each).

On top of that, some devices must also include additional information, according to the characteristics defined for them during the handshake process. For the HMD, if eye tracking is supported, two additional values, gX, and gY (double-precision *floats*, 8 bytes each), define the point at the screen at which the user is looking at, with (0,0) defined as the upper left corner and (1,1) as the lower right corner.

For the controllers, the buttons are listed in the order in which they were defined, followed by the joysticks, also in the order they were defined. For each button, if it's pushable, a *boolean* (1 byte) value indicates if the user is actually pushing it; if it's a trigger, a double-precision *float* (8 bytes) with a value between 0 and 1 indicates how far the trigger is being pushed. For each joystick, if it is pushable, a *boolean* (1 byte) value indicates if the user is actually pushing it; finally, two double-precision *floats* (8 bytes each) with a value between -1 and 1 indicates the X and Y coordinates, with (0,0) as the center position. The tracked devices only provide the ID and pose.

4 Remote rendering

The interaction of the different components that leads to the rendering, encoding, and transmission of every frame is depicted in Fig. 3.

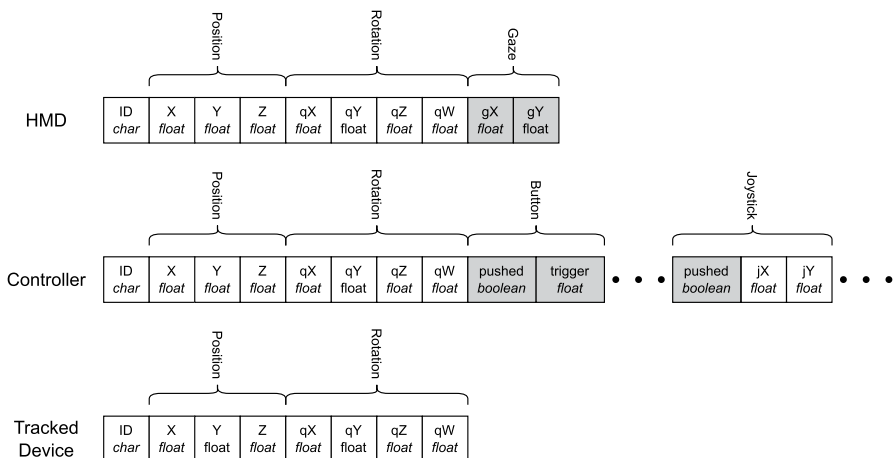


Fig. 2 Data sent from the Dummy VR Application to the Virtual Driver at the HMD's display frequency. Boxes in grey might not appear, depending on the characteristics of the devices defined during the handshake process

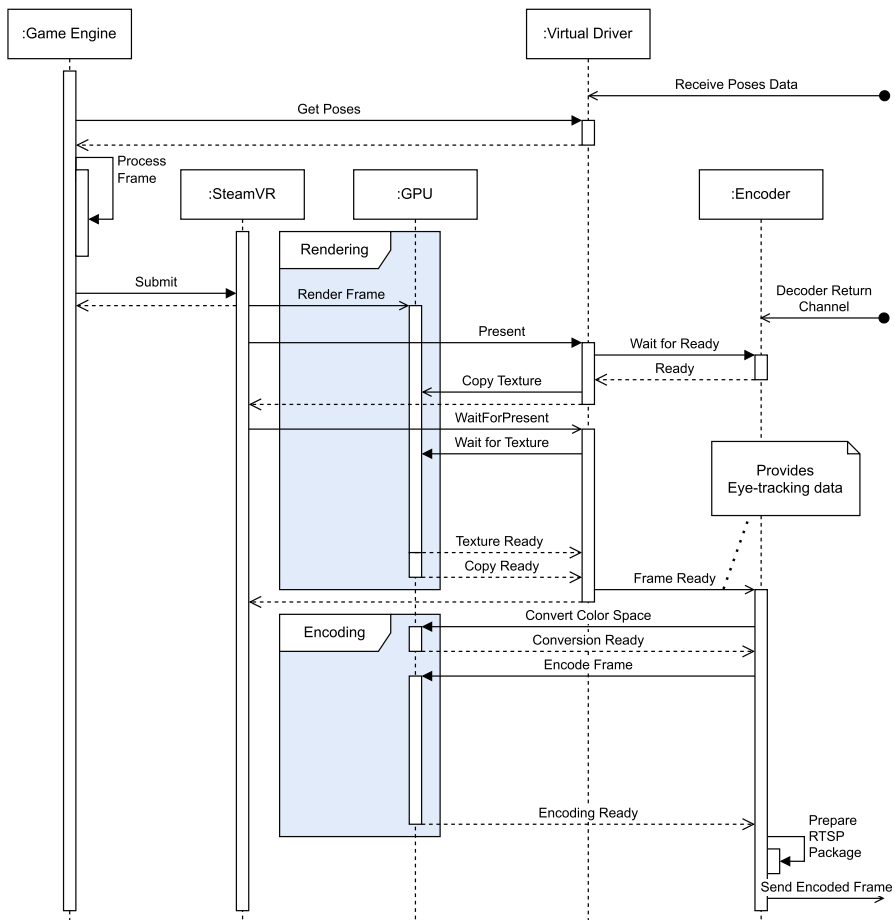


Fig. 3 Sequence diagram for the interaction between the game engine of the VR Application, SteamVR, the Virtual Driver and the Encoder. The lifetime marked as “GPU” indicates which tasks run on the GPU

For each frame, the VR Application’s game engine (running in the remote VR server) asks the active SteamVR driver for the most up to date information about all the connected hardware devices. With these data (poses, buttons pressed, and, potentially, user’s gaze), the game logic processes any changes that might have happened since the last frame was produced and passes to SteamVR the instructions needed to render a new frame using *Submit*. In NajVR the SteamVR driver is the Virtual Driver and all these devices’ data come from the client (see Fig. 1).

SteamVR enqueues into the GPU the rendering task for the new frame. Since this process takes some time, SteamVR then calls the *Present* and *WaitForPresent* functions of the Virtual Driver. *Present* should only perform non-blocking operations, like queueing GPU commands, and return as fast as possible. *WaitForPresent* can, on the other hand, actively wait until any pending task is finished.

Normally, a SteamVR driver would use the *WaitForPresent* function to draw the frame into the HMD's screen. The frame is provided as a single Direct3D 2D texture that contains both eyes' images, with the right one set horizontally after the left image ends. Since there is no physical HMD connected to the remote VR server, the Virtual Driver, instead of drawing the image, must prepare it for encoding. The *Present* function of the Virtual Driver:

1. Waits for the Encoder to be ready,
2. Acquires a mutex for accessing the 2D texture that the GPU is rendering,
3. Queues into the GPU a command for copying the texture to a different area in memory, so the original texture can be reused for a future frame, and
4. Releases the mutex.

The *WaitForPresent* function waits until the texture has been successfully copied by the GPU and then signals to the Encoder that a new frame is ready to be encoded. The Virtual Driver also provides the Encoder with a reference to the copied texture and the user's gaze data, which can be used to improve the encoding efficiency of some codecs, as will be explained later.

The first step of the encoding process is to perform a rescaling from the original rendering resolution to the target encoding resolution and a color conversion from RGB to YUV420. This is needed because the GPU generates images in RGB but YUV420 is the color space preferred by current video codecs, since it achieves a higher compression [16]. This conversion is performed by a CUDA kernel so it runs on the GPU.

The encoding itself can be performed by several video codecs. NajVR introduces LHE (described in Sect. 5), a codec designed for VR streaming, but support for other codecs (X264 and NVENC) has also been included for benchmarking and legacy reasons. Both X264 and NVENC are implementations of h264, the former by FFmpeg (version 155) and the latter by NVIDIA. All of them run on the GPU. The Encoder can be extended to use other codecs or different configurations of one of the already available ones if desired.

If the selected codec supports it, the eye-tracking data is used to encode the parts the user is actually looking at with higher quality. In the proposed system, only the LHE codec provides this feature, but new codecs could be implemented or an existing codec with this capability could be used [18, 19].

The Encoder sets up, using the Live555 open source library, a Real-Time Streaming Protocol (RTSP) server that streams every encoded frame to the client through Real-Time Transport Protocol (RTP) sessions[20]. If the frame was encoded using LHE, it uses the Network Abstraction Layer (NAL) separations of H264 nevertheless. This way, independent of the chosen codec, the established session appears as an H264 stream, reducing the complexity of the client Decoder.

To integrate the Live555 library into the Encoder, the *FramedSource* class, which determines a source of information per frame, was extended. The RTSP server runs in an independent thread, allowing the transmission to overlap with the rendering and encoding of each frame.

The Decoder receives the RTSP binary stream and produces video frames from it. The decoding process is less computationally expensive than the encoder process, so it can be run in more modest devices, even standalone HMDs. After decoding the frame, the decoder performs any modification that is needed for displaying it on the target hardware, filling the framebuffer so that the final image can be drawn on the next rendering tick.

The Decoder has an optional return channel that can be used to relay data (for example, packet loss metrics or other performance metrics) to the Encoder. Based on this information, the Encoder may modify its operation. This channel is also configured using UDP.

5 LHE codec

Logarithmic Hop Encoding (LHE) is a video codification and decodification algorithm that works in the spatial domain, based on observing the differences in luminance and chrominance within blocks of a frame by exploiting the Weber-Fresnel law. Its mathematical basis has been described in detail elsewhere [21] and its implementation details have been adapted and updated to suit the requirements of VR image encoding.

LHE has been designed to optimize for encoding time instead of bandwidth, aiming to reduce the motion-to-photon latency of a remote VR system. It is also very resistant to packet loss. It has been implemented using CUDA so that it can run on the GPU. A high-level block diagram of its encoding process can be seen in Fig. 4.

5.1 Perceptual relevance calculation and elastic downsampling

For each frame, the original image is divided into a grid of blocks of a set height and width, typically 40×40 pixels as this size is an exact divider of popular resolutions

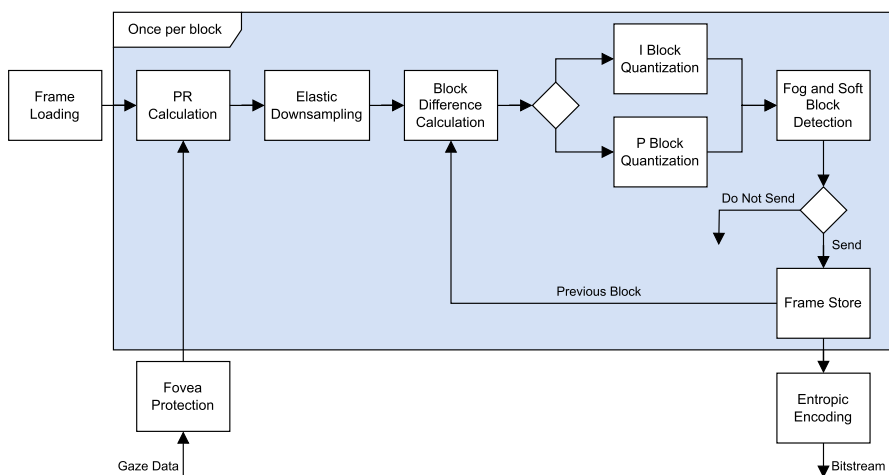


Fig. 4 Block diagram of the LHE encoding process' stages

(1920×1080 or 2880×1600 are just two examples). Each of these blocks is processed in parallel.

Before encoding, the blocks are downsampled to reduce the number of pixels without losing perceived quality. This is achieved by computing the Perceptual Relevance (PR) at the corners of each block, a value that ranges from 0 to 1. This magnitude quantifies the amount of variation of light intensity or color within a block in horizontal scanlines (PR_x) and in vertical columns (PR_y) and is used to establish a downsampling factor, which can be different in each direction; this concept is called elastic downsampling. A higher PR indicates a higher level of detail in that particular area.

LHE allows for different levels of aggressiveness when downsampling. Such aggressiveness is controlled through a QL (Quality Level) factor that can range between 0 and 99 and is combined with the computed PR_x and PR_y of the four corners of every block to end up with two compression factors, one horizontal and one vertical: the lower the QL, the higher the compression factors that will be assigned. The compression factors are then used to map every PR to the Plane Predicted Pixel values (PPP_x and PPP_y), which are the width and height in pixels, respectively, that the resulting downsampled block will have. For instance, a combination of PR_x and QL that produces a 2:1 downsampling ratio would result in a PPP_x of 20 horizontal pixels (from the original 40 per block).

The codec also uses the QL to perform bandwidth control. The PR calculation uses the bandwidth consumption of the last 7 frames. If it is above the target bandwidth by 10%, the QL is increased by one; if it is below it by 10%, the QL is decreased by one.

The result of this process is that blocks with higher PRs have more detail preserved; they experience less downsampling (or even no downsampling) than blocks with a low PR. This process is applied separately to the light intensity and the color information of the image.

Once the PPP calculation process is completed, the blocks are downsampled and used as input to the next stage.

5.2 Blocks processing

After downsampling, every block is classified either as an I block (Intra-coded, refreshment block) or a P block (Predicted, differential information block). This process is similar to the categorization of whole frames as I or P in other video codecs. In LHE, every frame can contain both I and P blocks, so the concept of a full I or P frame is not applicable.

In I blocks, the information is encoded spatially, by storing the difference between neighboring pixels; and in P blocks it is encoded temporally, by storing the differential data from that same block in the previous frame. Temporal encoding uses less bandwidth if the changes to the block are smoother but consumes more if the block's changes are sharper. To classify each downsampled block, it is compared with the same downsampled block of the previous frame.

Quantization is then performed differently for each type of block, but in both cases it involves two steps. First, every pixel value is predicted. Then the error between the predicted value and the real pixel value is calculated.

Since the encoding and decoding stages are symmetrical, this means that the only data that need to be sent to the Decoder are the errors between the predicted and real values, and not the values themselves.

The errors are encoded into “hops”: a limited set of possible values $H(x) = \{h_{N-1}, \dots, h_{-1}, h_0, h_1, \dots, h_N\}$, which are distributed logarithmically, as can be seen in Fig. 5. The actual value represented by each hop depends on the predicted value (h_0).

Even though the quantizer is forced to use a specific quantum (hop) from a logarithmic distribution, this choice of distribution makes the error introduced imperceptible. This is because the human eye is more sensitive to logarithmic changes in light intensity, even under high motion [22], a common circumstance in VR.

This quantization process is applied differently to I and P blocks, which has an impact on the parallelization factor. Since P blocks use temporal information from the previous frame’s block, the maximum degree of parallelization can be achieved: There are no dependencies between pixels inside the block and all pixels can be calculated at the same time. I block quantization is slower because parallelization is diagonal: Data from pixels above and to the left of each target pixel are used, so only pixels belonging to the diagonals of a block have independence and can be fully parallelized.

Again, the quantization process is applied separately to the luminance (light intensity) and chrominance (color) information of the block.

After quantization, the blocks in which no hops are farther from a threshold from the prediction (h_0) are marked as “soft blocks”. These are discarded to save bandwidth. For example, if the threshold is 2, blocks in which all hops are h_0 , h_1 or h_{-1} are discarded.

However, it has been experimentally observed that some of these soft blocks typically encode smoke, fog, or similar zones of slow variation between frames with a smooth movement. This continuous movement must be fluid, and, if some blocks are discarded, a characteristic choppiness (the movement freezes until a threshold is hit) can be observed. Therefore, if a soft block has a PR of less than 0.125 in any dimension (fog and smoke tend to have little detail) it is reclassified as a “fog block” and sent even if it was also marked as a soft block.

Finally, for every block, if it is identical to the same block from the previous frame, it is not sent.

Before delivering the bitstream to the RTSP server, hops are passed through one last module: the entropic encoder. This encoder translates all symbols (hops or com-

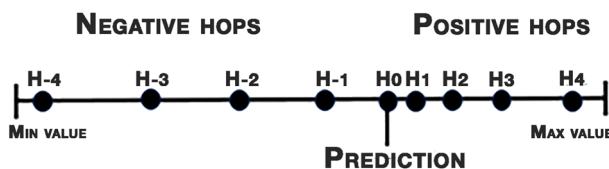


Fig. 5 Example of the value distribution of hops for a 9-hop system. As it can be appreciated, the distances between hops are asymmetrical: the positive hops are closer than the negative hops because the prediction (h_0) is closer to the maximum value

binations of hops) into binary form and compacts the information through statistical and prediction operations.

The decoding process starts with the depacketing phase, in which the received packet is disassembled, and the entropic decoder resolves the bitstream, obtaining the hops. The next step is the dequantizer, which produces the change from hops to light intensity and color signals. Every downsampled frame is stored so that it can be used for the P blocks of the next frame.

Using elastic interpolation, the image is reconstructed to its entire resolution based on the information provided by the Plane Predicted Pixels, which determine the downsampling ratio that was applied in each dimension.

Since each block is encoded independently, information about changes across blocks is not incorporated into the hop calculation. This tends to produce perceptible tears between blocks, so a deblocking filter was introduced in the decoder.

LHE does not use motion vectors nor I/P frames (it uses I/P blocks instead), so it has a high tolerance to lost packets; it is not necessary to wait for the next I frame. However, when a block is lost, the next block for that part of the image must be an I block, since P blocks cannot be updated if there is a lack of reference.

Due to this, LHE uses the return channel provided by the proposed architecture to indicate which blocks were lost and request that such blocks be classified as “I”. Blocks in every frame are numbered, starting with 0 and going from left to right and up to down. Therefore, a list of numbers to indicate the lost blocks is sent by the Decoder to the Encoder. This list does not include the frame in which the loss occurred, since the blocks should be classified as “I” as soon as possible. A number is not removed from the list until an I block is received for it.

5.3 Fovea protection

If the VR setup can track the user’s gaze, the Encoder can use this information to protect the area surrounding where the user is looking at, to preserve detail. These data, if available, are relayed to the Encoder, together with the frame being encoded.

The corner between blocks nearest to the coordinates of the user’s gaze is selected. This point acts as the virtual center of fovea protection. The size of the protected area can be configured to cover one, two, or up to three blocks in each dimension. This is shown in Fig. 6.

The PR of the corners of the protected blocks is always set to 1 (the maximum value). In this way, all the details of the protected blocks are preserved, and the blocks immediately surrounding them smoothly transition (they share corners with the protected blocks) to the PR of the rest of the image.

Figure 6 helps to visualize the effect of this fovea protection strategy. Since the encoded frame contains the images for the left and right eyes, two areas of the image are protected.

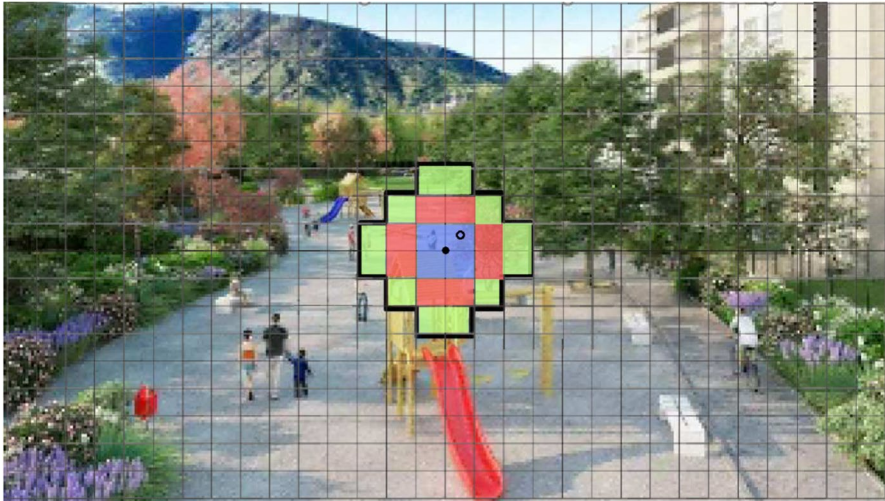


Fig. 6 Grid that shows the area surrounding the user's gaze (empty circle). This gaze is resolved to the nearest corner (black circle). One (blue blocks), two (red blocks), or three (green blocks) are protected in every dimension depending on the configuration. The protected blocks have their PR set to the maximum. The detail is fully preserved inside it. There is a smooth transition of resolution (through elastic downsampling and upsampling) that prevents the shape from being perceived

6 Tests & results

For the tests carried out in this section, two computers were used. The remote VR server is a laptop with an Intel Core i7-9850 H processor, 16 GB of RAM, and a NVIDIA QUADRO RTX 3000 graphics card. The local VR setup is comprised of an HTC Vive Pro Eye HMD, two HTC Vive Controllers 2.0, and an HTC Vive Tracker; all connected to a laptop with an Intel Core i5-9400 H processor, 8 GB of RAM, and a NVIDIA GTX 1050 Mobile graphics card that runs the Decoder and the Dummy VR Application. Both computers run Windows 10 Home 64-bit and use SteamVR version 1.8.21 as the VR runtime platform.

The HTC Vive Pro Eye HMD was selected due to its eye tracking capabilities. For obtaining the eye-tracking data, the HTC Vive SRanipal SDK version 1.3.3 was used.

At the time of writing this article, the GTX 1050 Mobile is an outdated graphics card, but it was selected for the client since its performance is comparable (20% slower) to the Qualcomm Adreno 740 mobile GPU present in the standalone Meta Quest 3 HMD. Therefore, the results obtained can be considered similar to those provided by commercially available standalone VR setups.

Two different network configurations were chosen to approximate a connection with high bandwidth (Ethernet or WiFi 6–7, as both standards support 1 Gbps or more) and a more resource-constrained environment (WiFi 5). For the high-bandwidth scenario, both computers were connected through a cabled local area network, using a NETGEAR ProSafe GS105 5-port Desktop Switch. For the resource-constrained scenario, the computers shared a WiFi 5 network provided by a TP-Link Archer C7 AC1750 Wireless Router.

To test the behaviour of NajVR on a WAN setup, we have deployed the server on the AWS (Amazon Web Services) eu-west-1 (Dublin, Ireland) location and run the client from a residential home (Madrid, Spain), connected to the Internet through a fiber-optic network advertised as 600 Mbps, using a ZTE F680 router. For this WAN setup, two tests were performed: With the client connected to the router either using a 1 Gbps Ethernet cable or a WiFi 5 network.

6.1 Codec comparison

The codec implementations tested were:

- LHE with 9 hops and a soft block threshold of 1.
- NVENC, NVIDIA's H264 implementation for ffmpeg.
- X264, the off the shelf implementation of H264 included in ffmpeg.

All implementations are running on the GPU. For NVENC and X264, two generic configurations are used for the comparison: A) MAX: Maximum efficiency, with a single I frame at the beginning of the stream. This configuration allows for maximum compression. B) AI: All intraframes, with no P frames.

Since LHE has no I or P frames (it has I or P blocks), only one configuration was used.

To compare the codecs, an approach based on matching the PSNR (Peak Signal-to-Noise Ratio) was chosen. This metric was selected because it is objective and all encoders tested support PSNR tuning, so the same value of PSNR can be chosen [23]. In this way, the performance of the codecs for the same image quality can be accurately compared.

10 video game 1920×1080 video sequences taken from the Xiph.org database, a popular source for evaluating compression technology, were used for the comparison. Since the video source for the proposed system is a game engine, this set seemed the most appropriate for the domain.

4 PSNR values (25, 27, 28, and 30 dB) were set and, for each of them, the average bandwidth consumed and the average encoding latency were obtained. Additionally, to better compare the quality of the encoded videos, MS-SSIM [24] was also used (see Fig. 7). These metrics mostly follow the quality/bandwidth relationship already present with PSNR, with LHE showing better results with MS-SSIM, pointing to the impact of perceived quality of its features.

Finally, for all codecs, some packets were purposely lost, to evaluate the effects of network problems on the image quality (reduction in PSNR).. The packet loss rates chosen were 0.5%, 1%, 5%, and 10%.

The results can be seen in Table 1. LHE is the codec with the lowest encoding time. Compared to both tested implementations of H264, LHE typically reduces the encoding latency by 50% and, in some cases, it can reach a reduction of up to 75%.

Concerning packet loss, LHE is considerably more resistant than the other codecs, experiencing less than 2 dB of image quality degradation even when 10% of the information is lost. Under the same circumstances, the H264 implementations experience a reduction of around 10 dB in PSNR. As expected, the AI configurations suffer

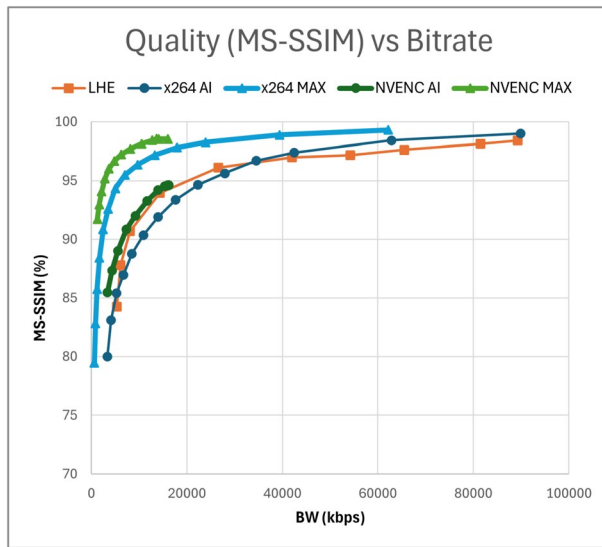


Fig. 7 Codec perceptual quality comparison tests using MS-SSIM

Table 1 Codec comparison tests

	25 dB	27 dB	28 dB	30 dB
Bandwidth (Mbps)				
<i>LHE</i>	16.4	25.5	40.1	69.6
<i>X264 MAX</i>	2.1	3.0	4.9	10.6
<i>X264 AI</i>	7.5	12.7	19.7	37.6
<i>NVENC MAX</i>	1.1	1.5	2.3	4.5
<i>NVENC AI</i>	4.8	8.1	13.6	29.4
Latency (ms)				
<i>LHE</i>	5.8	6.6	7.4	8.6
<i>X264 MAX</i>	13.5	13.7	14.1	14.8
<i>X264 AI</i>	12.4	12.8	13.3	14.1
<i>NVENC MAX</i>	11.5	11.4	11.5	11.4
<i>NVENC AI</i>	11.3	11.4	11.5	11.8
Effect on PSNR (dB) of packets lost				
<i>LHE</i>	−0.36	−0.54	−1.32	−1.86
<i>X264 MAX</i>	−5.48	−7.71	−10.9	−11.75
<i>X264 AI</i>	−1.97	−4.00	−6.57	−9.01
<i>NVENC MAX</i>	−5.85	−7.00	−9.93	−11.17
<i>NVENC AI</i>	−2.59	−2.59	−7.84	−9.24

the most. However, LHE is more bandwidth-hungry, necessitating between 2 and 15 times more bitrate for the same image quality.

We conducted an image-quality evaluation comparing the LHE codec without fovea protection and with fovea protection applied to the center of the frame (as if the user's gaze were focused there; a common scenario). The protected region covered

4% of the frame. In both configurations, we computed Structural Similarity (SSIM) [25] over the entire image and over only the protected region.

The test used two videos from the dataset: one depicting a video game with simple, block-like graphics, and another with photorealistic graphics. All encodes used a fixed Quality Level of 30. Results are reported in Table 2. As expected, the full-frame SSIM changes little because the protected area represents only 4% of the image. In contrast, SSIM within the protected (gaze) region improves markedly, consistent with SSIM's perceptual (nonlinear) sensitivity to localized quality. The photorealistic video shows a larger gain in regional SSIM, likely due to the greater amount of fine detail in the source content.

6.2 Latency analysis

To test the contribution of each part of the proposed solution to the overall round-trip latency, every component was set up so a timestamp of each start and end time was stored. 20 different configurations were tested. Each configuration results from the combination of one of the previously analyzed codecs (LHE, X264 MAX, X264 AI, NVENC MAX and NVENC AI) and one of the network configurations (high-bandwidth, resource-constrained, WAN cabled, and WiFi). All codecs were configured for a PSNR of 30 dB, since it's the threshold above which an image can be considered of good quality [26].

1,000 frames were processed for each configuration, and the times obtained were averaged. The results of the test can be seen in Fig. 8 and (note the differences in the Y axis). For the high-bandwidth setup, the total latency lies between 20–25 ms and, for the resource-constrained network, it lies between 25–40 ms.

These round-trip times could be increased by up to 11 ms due to the synchronization with the rendering tick of the client. This is the time between frames (90 Hz for most HMDs). At the extreme cases, a frame could be decoded and buffered just before it needs to be rendered (minimum delay: 0 ms of added waiting time), or just after the render process has finished (maximum delay: 11 ms of added waiting time).

It is important to note that the latency of the cloud location where the remote VR server would be placed has a big impact, as can be seen in Fig. 9. Therefore, using edge servers near users can significantly reduce latency by minimizing travel distance [27, 28]. About 60% of the users of cloud servers experience a median latency of 10 ms or less, and 90% of them of 25 ms or less [29, 30]. Putting all this together, when using an edge location for the remote VR server, a final round-trip latency between 30 and 75 ms can be expected, depending on the codec selected, the network quality, or the luck matching the rendering tick. Our own tests (between 50–60 ms) for both WAN setups confirm these numbers.

Table 2 SSIM comparison with and without fovea protection

Video source	W/o fovea prot.		W. fovea prot.		Change	
	SSIM	SSIM	SSIM	SSIM	SSIM	SSIM
	(image)	(fovea)	(image)	(fovea)	(image)	(fovea)
Simple graphics	0.8504	0.8207	0.8536	0.8712	+0.0032	+0.0505
Photorealistic graphics	0.7573	0.7332	0.7617	0.8012	+0.0044	+0.0680

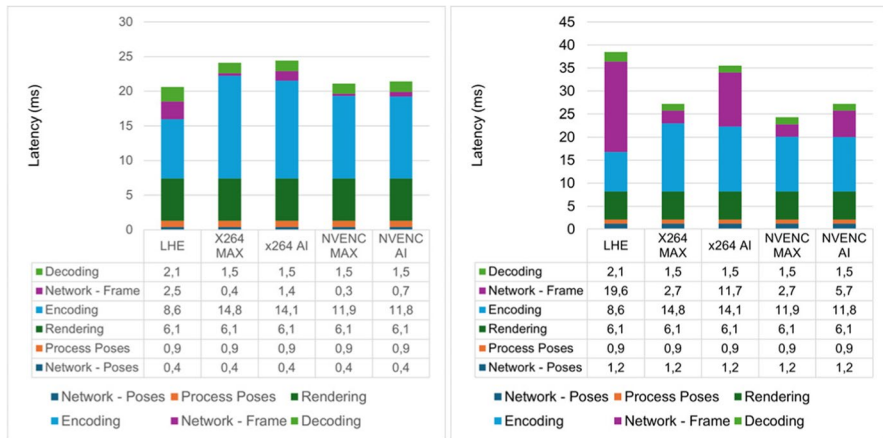


Fig. 8 Comparison of the combined round-trip latency of the proposed architecture with the codecs configured for a PSNR of 30 dB. The left plot shows the results for a high-bandwidth connection, while the right plot corresponds to a resource-constrained connection. Each bar represents the latency contribution of the individual processing steps

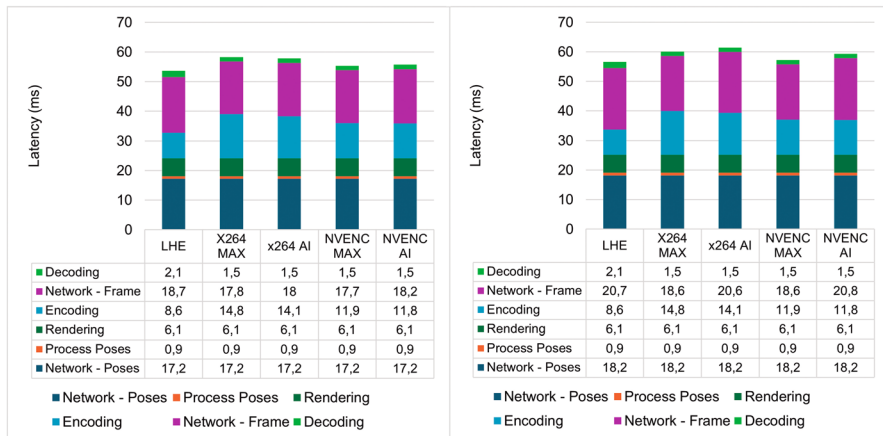


Fig. 9 Comparison of the combined round-trip latency of the proposed architecture with the codecs configured for a PSNR of 30 dB in a WAN deployment. The left plot shows the results when the client is connected to the router through Ethernet, and the right plot when connected using WiFi. Each bar represents the latency contribution of the individual processing steps

The results show that in environments with sufficient bandwidth the combined latency of LHE is the lowest, even considering the added transmission time. This suggests that with common residential bandwidths and using established cloud locations, the higher bandwidth required by LHE is not limiting. However, in situations where bandwidth is limited, the reduced LHE encoding time would likely be offsetted by a longer transmission time. It should be noted that the wireless test shows LHE as the codec with the lowest latency, probably because the residential network in which it

was tested was in a less saturated than the resource-constrained location tests, which were performed in a university building with many other wireless devices nearby.

We also evaluated end-to-end variability across wired and wireless links, including two WAN scenarios over the public Internet. With Ethernet, the standard deviation of latency ranged from 0.5 to 0.7 ms, with sporadic spikes of 7–10 ms. With WiFi, the standard deviation increased to 1.6–3.2 ms, with spikes of 40–80 ms. These results indicate that, beyond average delay, the principal risk on wireless links is that some packets arrive one frame late and are discarded. LHE's tolerance to packet loss should limit the visual impact of such events.

7 Conclusions & discussion

We have proposed NajVR: an architecture for remote rendering of off-the-shelf VR applications optimized for low latency that uses eye-tracking data to improve the encoding process.

NajVR uses LHE, a codec specifically designed for minimal latency and robustness, and implemented a version incorporating fovea protection. As shown in Table 1, LHE consistently achieves approximately half the latency of the commercial codecs against which it was compared, although at the cost of increased bandwidth consumption. This makes LHE the optimal choice for environments with enough bandwidth or where minimizing encoding latency is paramount. In contrast, NVENC is preferable in bandwidth-constrained situations; however, such environments are generally less suitable for latency-critical applications such as remote VR (see Figs. 8 and 9).

Recognizing that simply adding latency to local network simulations does not fully replicate a real-world client-to-cloud server connection, several various packet loss ratios were introduced to model connection instability and a WAN deployment was tested. This approach revealed LHE's superior performance in scenarios with significant packet loss. For example, even when faced with a substantial 10% packet loss, LHE maintained a higher PSNR than commercial codecs when experiencing only 0.5% packet loss (see Table 1).

LHE can use the user's fovea position to perform encoding with different levels of image quality. The computational complexity of the fovea protection module is negligible and in some cases can even speed up the encoding process, since fewer PRs need to be calculated. Adding fovea protection to other encoders may involve either reimplementations of the algorithm [18] or tweaks to existing implementation through parameters not related to the image, like the rate control (which indirectly controls the quality of the image) [19]. Although the latter solution does not involve rewriting the encoder, it adds a layer of complexity to bit-rate control.

Although our objective comparison of LHE with and without fovea protection shows a +0.0505 to +0.0680 SSIM gain within the protected region at the same Quality Level, establishing practical benefit requires human-subject evaluation. As future work, we will conduct controlled VR user studies to quantify subjective quality and QoE, in order to determine whether these SSIM gains translate into perceptible improvements for users. The fovea protection may have a high impact on the

QoE (Quality of Experience) of VR applications, since it impacts aspects like the readability of texts inside the virtual environment (for which all the details must be preserved) and tasks that require careful manipulation and sustained focus, in which the user is concentrated on a small part of the field of view.

When NajVR is used on a network with sufficient bandwidth and low latency to connect to the server, our tests show that even when running on modest hardware, it has a latency of just over 20 ms (see Fig. 8(left)) in a LAN and just over 50 ms (see Fig. 9(left)) in a WAN. Considering the additional latency that could come from synchronization with the HMD's rendering tick, and the use of a nearer edge server (for instance, in the same city), the total latency would be between a best-case scenario of 30 ms and a worst-case scenario of 75 ms.

Although a motion-to-photon latency of 20 ms has long been considered the maximum acceptable for VR applications [31], recent human-subject studies challenge this conventional wisdom. For example, a study [32] discovered that image quality is the predominant factor influencing subjective Quality of Experience (QoE). The authors found that a latency of up to 100 ms had minimal to no impact on slow-paced games, with an appreciable but still tolerable effect on fast-paced games. Furthermore, their work highlighted that high packet loss rates negatively affect QoE more significantly than latency. Another study [30], using a fast-paced shooter game, reported that QoE was most degraded by latencies of 75 ms or higher, with jitter causing even greater subjective quality deterioration. These findings align with another survey [33] of both slow and fast-paced VR games, which concluded that users could barely discern differences in latency values up to 90 ms. Furthermore, a separate study [34] identified motion tracking accuracy as the most influential factor in user performance in VR experiences.

Collectively, these studies suggest that users can maintain immersion in virtual environments even with relatively high latencies (up to 75–100 ms). However, variations in latency (jitter) significantly impact QoE by forcing users to “recalibrate” their senses. Our tests have indicated that this jitter is more present in wireless networks. LHE and its high resistance to packet loss makes it more tolerant to jitter, making it a good choice for environments in which this problem is more frequent.

The latency numbers reported here could be improved without changes to the architecture described in Sect. 2 or to its implementation. For example, NVIDIA's Ada Lovelace GPUs deliver approximately 24% higher encoding throughput than the Turing devices used in our setup [35]. Latency can also be reduced by adopting more advanced access technologies, since remote VR is expected to benefit from the lower latency and higher bandwidth of current 5 G and forthcoming 6 G networks; nevertheless, mitigating jitter typical of widely deployed 5 G deployments remains essential [30]. Other work has explored terahertz [36] and mmWave [37] links, the latter leveraging HMD pose information to maintain line-of-sight connectivity required by this technology.

Beyond hardware and network upgrades, several software-level optimizations can further decrease motion-to-photon latency. Short-horizon motion prediction and pose extrapolation within the Virtual Driver (on the order of tens of milliseconds) can help mask transport delay [38, 39]. Avoiding unnecessary conversion between RGB and YUV (or performing them in-GPU with zero-copy paths) can also reduce per-

frame processing, though at a potential bandwidth penalty, which warrants an explicit bandwidth–latency trade-off analysis [16]. Techniques such as asynchronous reprojection and late latching ensure the most recent tracking data is used at display time. Finally, hybrid cloud–edge prediction architectures, where nearby edge nodes correct or refine cloud-rendered frames, can combine the scalability of remote computation with the responsiveness of local processing.

Taken together, these measures can reduce end-to-end latency beyond the 30–75 ms observed in real-world settings and, under favorable hardware and network conditions, approach the commonly cited 20–25 ms range, bringing remote VR performance closer to that of locally rendered experiences.

NajVR prioritizes low end-to-end latency over high-throughput links, where inserting additional compression stages typically adds processing delay (extra encode/decode work, adaptation buffers, and possible GPU/CPU contention). Nevertheless, in bandwidth-constrained settings (e.g., congested Wi-Fi or limited uplinks), the pipeline can accommodate optional remedies without altering its core design. An adaptive compression module between the encoder and streamer can reduce bitrate via online entropy coding and, when latency budgets allow, content-aware dictionaries learned offline or during a brief warm-up. Hybrid strategies can also switch between LHE and standard GPU codecs (e.g., H.264/AV1) and tighten ROI-aware quantization in the periphery to relieve bandwidth while preserving VR latency tolerances. Because universal dictionaries seldom generalize across content, these mechanisms should be policy-driven and activated only under detected bandwidth pressure. A systematic evaluation of trigger thresholds, content dependence, and the net impact on motion-to-photon delay is left for future work.

Finally, it should be noted that the architecture proposed could, with very few modifications (using a live feed as a video source instead of game engine-generated images), be used in domains other than gaming or entertainment, to implement an immersive remote work system. The applications of such a system could range from remote consultation [40], telesurgery [41], or remote operation in dangerous environments [42], among others.

Acknowledgements We want to thank Sergio Molina Cardin for his help testing the codecs.

Author contributions R.G.C, J.J.G.A, and A.O. conceptualized the overall idea, led the research, and obtained funding. J.J.G.A, M.A.G. and F.J.J.Q. developed the LHE codec. J.A.S.P., P.P.T., R.G.C and A.O. developed the remaining components. All authors tested the proposed system and participated in the writing of the manuscript.

Funding This work was supported by the Spanish Ministry of Science, Innovation and University, under the grant European Regional Development Fund (ERDF) with project references IDI-20191120 and PID2021-127096OB-I00.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Cuervo E, Wolman A, Cox LP, Lebeck K, Razeen A, Saroiu S, Musuvathi M (2015) Kahawai: high-quality mobile gaming using GPU offload. In: Proceedings of the 13th annual international conference on mobile systems, applications, and services, pp. 121–135. <https://doi.org/10.1145/2742647.2742657>
2. So RH, Griffin MJ (1992) Compensating lags in head-coupled displays using head position prediction and image deflection. *J Aircr* 29(6):1064–1068. <https://doi.org/10.2514/3.46285>
3. Kim J, Knowles P, Spjut J, Boudaoud B, McGuire M (2020) Post-render warp with late input sampling improves aiming under high latency conditions. *Proc ACM Comput Graph Interact Tech* 3(2):1–18. <https://doi.org/10.1145/3406187>
4. Mark WR, McMillan L, Bishop G (1997) Post-rendering 3D warping. In: Proceedings of the 1997 symposium on interactive 3D graphics, p. 7. <https://doi.org/10.1145/253284.253292>
5. Van Waveren JMP (2016) The asynchronous time warp for virtual reality on consumer hardware. In: Proceedings of the 22nd ACM conference on virtual reality software and technology, pp. 37–46. <https://doi.org/10.1145/2993369.2993375>
6. Lai Z, Hu YC, Cui Y, Sun L, Dai N (2017) Furion: engineering high-quality immersive virtual reality on today's mobile devices. In: Proceedings of the 23rd annual international conference on mobile computing and networking, pp. 409–421. <https://doi.org/10.1145/3117811.3117815>
7. Kämäräinen T, Siekkinen M, Eerikainen J, Ylä-Jääski A (2018) CloudVR: cloud accelerated interactive mobile virtual reality. In: Proceedings of the 26th ACM international conference on multimedia, pp. 1181–1189. <https://doi.org/10.1145/3240508.3240620>
8. Zhou Y, Popescu V (2024) Clovr: fast-startup low-latency cloud VR. *IEEE Trans Vis Comput Graph*. <https://doi.org/10.1109/TVCG.2024.3372059>
9. Kravec J, Káčerik M, Bittner J (2023) Pvli: potentially visible layered image for real-time ray tracing. *Vis Comput* 39(8):3359–3372. <https://doi.org/10.1007/s00371-023-03007-5>
10. Cordeiro Barboza D, Muchaluat-Saade DC, Gonzales ECW, Gimenez Passos D (2022) An architecture for multi-layer object coding in 2d game streaming using shared data in a multi-user environment. *Entertain Comput* 42:100490. <https://doi.org/10.1016/j.entcom.2022.100490>
11. Stengel M, Majercik Z, Boudaoud B, McGuire M (2021) A distributed, decoupled system for losslessly streaming dynamic light probes to thin clients. In: Proceedings of the 12th ACM multimedia systems conference, pp. 159–172. <https://doi.org/10.1145/3458305.3463379>
12. Tan YW, Low SE, Chow J, Teo J, Bhojan A (2024) Dhr+ s: distributed hybrid rendering with realistic real-time shadows for interactive thin client metaverse and game applications. *Vis Comput*. <https://doi.org/10.1007/s00371-024-03501-4>
13. Shi S, Gupta V, Hwang M, Jana R (2019) Mobile VR on edge cloud: a latency-driven design. In: Proceedings of the 10th ACM multimedia systems conference, pp. 222–231. <https://doi.org/10.1145/3304109.3306217>
14. Mehrabi A, Siekkinen M, Kämäräinen T, Ylä-Jääski A (2021) Multi-Tier CloudVR: leveraging edge computing in remote rendered virtual reality. *ACM Trans Multimedia Comput Commun Appl (TOMM)* 17(2):1–24. <https://doi.org/10.1145/3429441>
15. Liu L, Zhong R, Zhang W, Liu Y, Zhang J, Zhang L, Gruteser M (2018) Cutting the cord: designing a high-quality untethered VR system with low latency remote rendering. In: Proceedings of the 16th annual international conference on mobile systems, applications, and services, pp. 68–80. <https://doi.org/10.1145/3210240.3210313>

16. Rohloff A, Allen Z, Lin K-M, Okrend J, Nie C, Liu Y-C, Tseng H-W (2021) OpenUVR: an open-source system framework for untethered virtual reality applications. In: 2021 IEEE 27th real-time and embedded technology and applications symposium (RTAS), pp. 223–236. <https://doi.org/10.1109/RTAS52030.2021.00026>
17. Xu S (2024) Application of video adaptive sampling based on soft computing in higher education evaluation and e-learning interactive environment. *Entertain Comput* 51:100750. <https://doi.org/10.1016/j.entcom.2022.100490>
18. Frieß F, Braun M, Bruder V, Frey S, Reina G, Ertl T (2020) Foveated encoding for large high-resolution displays. *IEEE Trans Vis Comput Graph* 27(2):1850–1859. <https://doi.org/10.1109/TVCG.2020.3030445>
19. Illahi GK, Gemert TV, Siekkinen M, Masala E, Oulasvirta A, Ylä-Jääski A (2020) Cloud gaming with foveated video encoding. *ACM Trans Multimedia Comput Commun Appl (TOMM)* 16(1):1–24
20. Mehmood E, Anees T (2020) Challenges and solutions for processing real-time big data stream: a systematic literature review. *IEEE Access* 8:119123–119143. <https://doi.org/10.1109/ACCESS.2020.3005268>
21. García Aranda JJ, Alarcón Granero M, Juan Quintanilla FJ, Caffarena G, García-Carmona R (2021) Elastic downsampling: an adaptive downsampling technique to preserve image quality. *Electronics* 10(4):400. <https://doi.org/10.3390/electronics10040400>
22. Zanker JM (1995) Does motion perception follow Weber's law? *Perception* 24(4):363–372. <https://doi.org/10.1068/p240363>
23. Laude T, Adhisantoso YG, Voges J, Munderloh M, Ostermann J (2019) A comprehensive video codec comparison. *APSIPA Trans Signal Inf Process* 8:30. <https://doi.org/10.1017/ATSIP.2019.23>
24. Wang Z, Simoncelli EP, Bovik AC (2003) Multiscale structural similarity for image quality assessment. The thirty-seventh asilomar conference on signals, systems & computers, vol 2. IEEE, pp 1398–1402
25. Wang Z, Bovik AC, Sheikh HR, Simoncelli EP (2004) Image quality assessment: from error visibility to structural similarity. *IEEE Trans Image Process* 13(4):600–612. <https://doi.org/10.1109/TIP.2003.819861>
26. Baaziz N, Zheng D, Wang D (2011) Image quality assessment based on multiple watermarking approach. In: 2011 IEEE 13th international workshop on multimedia signal processing, pp. 1–5. <https://doi.org/10.1109/MMSP.2011.6093787>
27. Swain CK, Shankar R, Sahu A (2024) Edge data distribution as a network steiner tree estimation in edge computing. *Comput* 106:1585–1609. <https://doi.org/10.1007/s00607-024-01259-0>
28. Makris A, Psomakelis E, Korontanis I et al (2024) Edge-driven Docker registry: facilitating XR application deployment. *Computing* 106:3479–3501. <https://doi.org/10.1007/s00607-024-01310-0>
29. Charyyev B, Arslan E, Gunes MH (2020) Latency comparison of cloud datacenters and edge servers. In: GLOBECOM 2020–2020 IEEE global communications conference, pp. 1–6. <https://doi.org/10.1109/GLOBECOM42002.2020.9322406>
30. Rossi HS, Mitra K, Larsson S, Åhlund C, Cotanis I (2024) Subjective QoE assessment for virtual reality cloud-based first-person shooter game. In: ICC 2024–IEEE international conference on communications, pp. 4698–4703. <https://doi.org/10.1109/ICC51166.2024.10622467>
31. Abdallah M, Griwodz C, Chen K-T, Simon G, Wang P-C, Hsu C-H (2018) Delay-sensitive video computing in the cloud: a survey. *ACM Trans Multimedia Comput Commun Appl (TOMM)* 14(3s):1–29. <https://doi.org/10.1145/3212804>
32. Lee K-Y, Singla A, Cesar P, Hsu C-H (2024) Adaptive cloud VR gaming optimized by gamer QoE models. *ACM Trans Multimedia Comput Commun Appl*. <https://doi.org/10.1145/3680551>
33. Li Y-C, Hsu C-H, Lin Y-C, Hsu C-H (2020) Performance measurements on a cloud VR gaming platform. In: Proceedings of the 1st workshop on quality of experience (QoE) in visual multimedia applications, pp. 37–45. <https://doi.org/10.1145/3423328.3423497>
34. Pan Y (2024) Sports game teaching and high precision sports training system based on virtual reality technology. *Entertain Comput* 50:100662. <https://doi.org/10.1016/j.entcom.2024.100662>
35. Arunruangsirilert K, Katto J (2024) Evaluation of hardware-based video encoders on modern gpus for uhd live-streaming. In: 2024 33rd International conference on computer communications and networks (ICCCN), pp. 1–9. <https://doi.org/10.1109/ICCCN61486.2024.10637525>
36. Ma Y, Ota K, Dong M (2024) QoE optimization for virtual reality services in multi-RIS-assisted terahertz wireless networks. *IEEE J Sel Areas Commun*. <https://doi.org/10.1109/JSAC.2023.3345394>
37. Elbamby MS, Perfecto C, Bennis M, Doppler K (2018) Toward low-latency and ultra-reliable virtual reality. *IEEE Network* 32(2):78–84. <https://doi.org/10.1109/MNET.2018.1700268>

38. Chen H, Wei L, Liu H, Shi B, Zhang G, Zha H (2022) MOUNT: learning 6DoF motion prediction based on uncertainty estimation for delayed AR rendering. *IEEE Trans Vis Comput Graph*. <https://doi.org/10.1109/TVCG.2022.3228807>
39. Bao Y, Wu H, Zhang T, Ramli AA, Liu X (2016) Shooting a moving target: motion-prediction-based transmission for 360-degree videos. In: 2016 IEEE International conference on big data (big data), pp. 1161–1170. <https://doi.org/10.1109/BigData.2016.7840720>
40. García FM, Moraleda R, Schez-Sobrinho S, Monekosso DN, Vallejo D, Glez-Morcillo C (2023) Health-5G: a mixed reality-based system for remote medical assistance in emergency situations. *IEEE Access* 11:59016–59032. <https://doi.org/10.1109/ACCESS.2023.3285420>
41. Acemoglu A, Kriegelstein J, Caldwell DG, Mora F, Guastini L, Trimarchi M, Vinciguerra A, Carobbio ALC, Hysenbelli J, Delsanto M et al (2020) 5G robotic telesurgery: remote transoral laser microsurgery on a cadaver. *IEEE Trans Med Robot Bionics* 2(4):511–518. <https://doi.org/10.1109/TMRB.2020.3033007>
42. Szczurek KA, Prades RM, Matheson E, Rodriguez-Nogueira J, Di Castro M (2023) Multimodal multi-user mixed reality human-robot interface for remote operations in hazardous environments. *IEEE Access* 11:17305–17333. <https://doi.org/10.1109/ACCESS.2023.3245833>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

José A. Santos-Paz¹ · José J. García Aranda² · Manuel Alarcón Granero³ · Francisco J. J. Quintanilla⁴ · Pablo Perez-Tirador¹ · Abraham Otero¹ · Rodrigo García-Carmona⁵

✉ Rodrigo García-Carmona
rodrigo.garcia@upm.es

José A. Santos-Paz
jose.santospaz@ceu.es

José J. García Aranda
jose_javier.garcia_aranda@nokia.com

Manuel Alarcón Granero
malarcong@indra.es

Francisco J. J. Quintanilla
francisco_jose.juan@seg-social.es

Pablo Perez-Tirador
pablo.pereztirador@ceu.es

Abraham Otero
aotero@ceu.es

¹ Department of Information Technologies, Institute of Technology, Universidad San Pablo-CEU, CEU Universities, Campus Montepríncipe, Avenida Montepríncipe, 28660 Boadilla del Monte, Madrid, Spain

² R&D Department, Nokia Spain, Maria Tubau 9, 28050 Madrid, Madrid, Spain

³ Indra Sistemas, Av. de Castilla 2, 28830 Torrejón de Ardoz, Madrid, Spain

⁴ Gerencia de Informática, Seguridad Social, Dr. Tolosa Latour, 28041 Madrid, Madrid, Spain

⁵ Information Processing and Telecommunications Center (IPTC), Universidad Politécnica de Madrid, ETSI Telecomunicación, Avda. Complutense 30, 28040 Madrid, Madrid, Spain